

Ciao, sto seguendo questo articolo per testare un po' git e github: <https://www.baeldung.com/vaadin>

Esercitazione GIT e GITHUB

- Sviluppatori:

- ebemarco di Reply
- marcoBe1t99 è il mio personale

Progetto:

- *Frontend*: Vaadin →

Accedi

Registrati gratuitamente

= ChatGPT ▾

Mostra di più ▾

Mi sembra un buon esercizio per simulare un piccolo progetto con due sviluppatori e un project manager. L'idea di usare due account GitHub diversi è utile anche per prendere confidenza con fork, pull request e code review.

Ti suggerirei di rendere l'esercizio un po' più realistico, simulando un normale flusso Git/GitHub.

Struttura del progetto

- **Repository principale:** vaadin-demo
- **Project Manager:** ebemarco
- **Developer Frontend:** ebemarco
- **Developer Backend:** marcoBe1t99

Stack:

- Frontend: Vaadin 24
- Backend: Spring Boot
- Database: PostgreSQL

Fase 1 - Creazione repository

Sul PC del project manager:

</> Bash



```
mkdir vaadin-demo
cd vaadin-demo

git init
```

Crea il progetto Spring Boot + Vaadin (seguendo l'articolo).

Primo commit:

```
</> Bash

git add .
git commit -m "Initial Spring Boot + Vaadin project"
```

Pubblica su GitHub:

```
</> Bash

git remote add origin https://github.com/ebemarco/vaadin-demo.git
git push -u origin main
```

Fase 2 - Lo sviluppatore Backend

Con l'account marcoBelt99 :

```
</> Bash

git clone https://github.com/ebemarco/vaadin-demo.git
```

oppure

Fork

↓

Clone del fork

↓

Aggiunta del repository originale come upstream:

```
</> Bash

git remote add upstream https://github.com/ebemarco/vaadin-demo.git
```

Fase 3 - Creazione dei branch

Manager:

</> Bash



```
git checkout -b frontend
git push origin frontend
```

Backend:

</> Bash



```
git checkout -b backend
git push origin backend
```

Così ogni sviluppatore lavora nel proprio branch.

Fase 4 - Frontend

ebemarco

Crea

ExampleLayout.java



Commit

</> Bash



```
git add .
git commit -m "Add ExampleLayout view"
git push origin frontend
```

Apri una Pull Request

frontend

↓

main



Il manager la approva e la unisce.

Fase 5 - Backend

marcoBelt99

Nel branch backend crea ad esempio:

```
model/  
  
service/  
  
repository/  
  
controller/
```



con un'entità semplice, ad esempio

```
</> Java  
  
Person
```



Commit

```
</> Bash  
  
git add .  
git commit -m "Add Person entity"  
git push origin backend
```



Pull Request verso main .

Fase 6 - Aggiornamento del backend

Dopo che il frontend è stato unito, il backend si aggiorna:

```
</> Bash  
  
git checkout backend  
git fetch upstream  
git merge upstream/main
```



oppure

```
</> Bash  
  
git pull upstream main
```



Questo simula il lavoro quotidiano.

Fase 7 - Simulare un conflitto

Entrambi modificano lo stesso file, ad esempio

ExampleLayout.java



Frontend:

</> Java



```
add(new H1("Homepage"));
```

Backend:

</> Java



```
add(new H1("Welcome"));
```

Quando il backend esegue

</> Bash



```
git merge main
```

Git produrrà un conflitto da risolvere manualmente.

Fase 8 - Tag di rilascio

Dopo il merge:

</> Bash



```
git tag v1.0  
git push origin v1.0
```

Fase 9 - Nuova feature

Creazione del branch:

</> Bash



```
git checkout -b feature/login
```

Sviluppo della feature.

Merge.

Eliminazione del branch:

</> Bash



```
git branch -d feature/login  
git push origin --delete feature/login
```

Comandi Git che avrai occasione di usare

</> Bash



```
git init  
git clone  
git status  
git add  
git commit  
git log  
git diff  
git branch  
git checkout  
git switch  
git merge  
git rebase  
git fetch  
git pull  
git push  
git stash  
git tag  
git remote
```

Cosa imparerai con questa simulazione

- Creazione e pubblicazione di un repository GitHub.
- Lavoro su branch separati per frontend e backend.
- Gestione di origin e upstream (utile nel caso di fork).
- Apertura e revisione di Pull Request.
- Risoluzione di conflitti di merge.
- Creazione di tag per i rilasci.

- Simulazione di un flusso di lavoro tipico di un team che utilizza Git e GitHub.

È un esercizio che riproduce abbastanza fedelmente un workflow professionale basato su feature branch e Pull Request.

